

The University of Saskatchewan

Saskatoon, Canada

Department of Computer Science

CMPT 306– Game Mechanics

Assignment 2

Date Due: November 7, 2025, 11:59pm

Total Marks: 60

This assignment is technically due Friday November 7, 2025, but late submissions will be accepted until Sunday November 9 at 11:59pm with no penalty deducted.

General Instructions

- Assignments must be submitted using Canvas.
- Unless otherwise specified, programs should be written in GDScript and be compatible with Godot 4.4 or newer.
- Unless a specific framework or library is requested, all submissions should be compatible with the basic vanilla version of the engine.
- In some assignments, certain features in Godot will be limited or forbidden, usually because they circumvent the learning objectives of the assignment. The assignment will specifically state this where applicable, but ensure you are following the assignment instructions carefully.
- VERY IMPORTANT: Canvas is very fragile when it comes to submitting multiple files. We insist that you package all of the files for all questions for the entire assignment into a ZIP archive file. This can be done with a feature built into the Windows explorer (Windows), or with the zip terminal command (LINUX and Mac). We cannot accept any other archive formats. This means no tar, no gzip, no 7zip, no RAR. Non-zip archives will not be graded. We will not grade assignments if these submission instructions are not followed.
- Unless the assignment specifically asks for the entire Godot project, please only submit the files requested. The markers will have their own marking environment where they will drop your files into, if they have to navigate through your project structure they will deduct marks.
- It is expected that all assignment work is under git version control. Each assignment will have different expectations and will list them explicitly. A portion of each assignment marks will be a submitted git log that demonstrates that you followed the required development process.

Question 1 (10 points):

The purpose of this assignment is to practice using git with Godot, and demonstrate proper development workflows.

Your Task

In a previous tutorial you should have set up a git repository and practiced using it with Godot. It is up to you whether you create a new repository for each assignment or manage subdirectories for assignments. The important part is you are using git and follow the guidelines described below.

This assignment contains 1 question, with the following git expectations:

Question 2 : There is no required sequence of steps or minimum number of commits to receive full marks for this question. The following will be considered when evaluating your submissions:

- Is the gitlog legitimate? If the format looks suspicious or tampered with, the marker reserves the right to request access to your repository to verify the git log matches the repository's history. **Fabrication of git logs is considered an act of academic misconduct and is a serious offence**
- Commit messages should be detailed and useful. Each commit should be for a single change, bug fix, feature and be accompanied with a message describing what was done. The point of commit messages is to have a record of changes over time.
- The entire git log should tell the story of your assignment implementation, and demonstrate proper iterative development, as described in lectures and from Assignment 1.

Evaluation

10 marks : Git log demonstrates an iterative process, following guidelines described above.

-2 marks : git log contains the entire git history

Provided Files

The instructor has provided no files for this question.

Files to Hand In

Include in your final submission a file named `gitlog.txt` containing all commits pertaining to this assignment. Marks will be deducted if the entire git history is submitted. There are various ways to restrict the commits to those from this specific assignment, such as tagging the repository at the beginning and only including commits made after the tag. Alternatively, you could create a separate branch for each assignment and submit the git log for that particular branch.

Question 2 (50 points):

The purpose of this question is to practice a variety of old and new skills and implement a tic-tac-toe game.

Your Task

For this question you will be implementing a version of the game tic-tac-toe (if you are unfamiliar, feel free to briefly research the game to familiarize yourself with the rules and procedures).

Your submitted game should have the following features:

- A UI Label at the top left of the window indicating whose turn it currently is (i.e., X or O)
- The board should have dimensions of 3x3
- The beginning state of the board should be all tiles unclaimed or empty (i.e., neither X nor O)
- Clicking on an unclaimed tile should transform it into the current player's symbol (i.e., X or O)
- When a valid move is made, the turn should switch to the other player
- Clicking on a claimed tile should do nothing
- Clicking outside of the board should do nothing
- When a player wins (i.e., a row of 3 matching symbols) a message should be displayed indicating the winner, and the option to start a new game.
- When the board becomes full and no winner detected, a message should be displayed indicating that the 'cat' won

In addition to the above required features, your code should be organized into a model-view-controller architecture (MVC). You should be familiar with MVC from previous courses, but implementing it in the context of a game engine is likely new. Below are some general tips and guidelines:

- The Controller should register user input (in this case, the user clicking on a tile), interpret that input, and send the appropriate message(s) to the model
- The Model class(es) should contain the underlying data model of the game, as well as basic game logic (as directed by messages from the Controller). Whenever the model changes, it should broadcast an update to the View(s) using an implementation of the Observer pattern
- The view should handle the visual representation of the game (in this case, a `TileMapLayer` node)
- The observer pattern can be implemented in Godot using signals to broadcast a message, and other nodes/scripts can connect to 'catch' the signal each time it is emitted.
- The game and logic is simple, but the architecture will take some time to figure out and implement correctly.

Evaluation

5 marks : Complete UML diagram describing system

5 marks : Reasonable error handling: clicking on occupied square, clicking outside of board grids, etc. handled correctly

10 marks : Documenting functions using documenting style discussed in lecture and on previous assignments

10 marks : Adequate unit tests conducted using GUT, where appropriate

10 marks : Generally follows MVC software architecture

10 marks : Game works correctly, as described in assignment specification

Provided Files

A spritesheet for tile states is provided by instructor.

Files to Hand In

- UML diagram illustrating the final architecture of your game
- Complete Godot project

Note: While the UML diagram is only worth 5 marks, other parts of the Evaluation will use the UML diagram as part of marking. If the UML diagram is not submitted, marks may be deducted from MCV architecture in the event the marker has trouble determining how your code is organized.

Files Provided

A spritesheet of tic-tac-toe states is provided.

What to Hand In

Your submission should contain the following files:

gitlog.txt : gitlog indicating development process for the assignment, as indicated by Question 1.

Godot Project : the complete project directory should be included, and be compatible with the required version of Godot.

UML diagram demonstrating the way your game is organized.

Files should be submitted in a .zip archive file, with the naming convention: a1-nsid.zip, where NSID is replaced with your specific NSID (i.e., a1-abc123.zip).

Grading Rubric

The grading rubric can be found on Canvas.